SOFTWARE REQUIREMENTS SPECIFICATION

VERSION 3.0 — FINAL

Blog Monitor & Automated Content Summarization System

Document	
Project Title	Blog Monitor & Automated Content Summarization System
Version	3.0 — FINAL (supersedes v1.0 and v2.0)
Prepared By	[Your Name] — .NET Intern Developer
Reviewed By	[Manager Name] — Engineering Manager
Date	June 2025
Status	FINAL — Ready for Development
Tech Stack	React.js | ASP.NET Core .NET 10 | SQL Server | DuckDB | Playwright

✓ **All previous versions superseded**

This document incorporates all manager decisions from v2.0 AND all architectural improvements identified through peer review. The pipeline has been redesigned with hash-based change detection as the single source of truth across all URL types.

1. What This Document Is

This is the final Software Requirements Specification for the Blog Monitor & Automated Content Summarization System. It was written by the intern developer based on requirements narrated verbally by the Engineering Manager, then refined through two rounds of architectural review.

This document is designed to serve three purposes at once:

- A submission to your Engineering Manager explaining the full project scope and design decisions.
- A development reference that maps every feature to a concrete implementation plan.
- A GitHub Copilot context file — upload this document to your Copilot workspace so the AI assistant can give accurate, project-specific code suggestions.

> **Why version 3.0?**
> v1.0 captured the initial verbal requirements. v2.0 resolved all ten open questions with manager decisions. v3.0 fixes four architectural loopholes found during peer review — specifically around hash-based change detection, crawl limits, and content cleaning order. The core product idea has not changed.

2. Project Overview — Plain English

2.1 What the system does

A client registers a website or blog URL through a web interface. From that point, the system runs automatically every day:

1. It finds all the pages on that website (using the site's sitemap file, or by crawling the homepage if no sitemap exists).
2. For each page, it downloads the content, strips out everything that is not the actual article text (ads, menus, cookie banners, etc.), and takes a fingerprint of what remains.
3. It compares today's fingerprint against yesterday's. If nothing changed, it logs that and moves on. If something changed or a new page appeared, it summarises the new content.
4. Every Sunday, it compiles everything from the whole week into a single neat report and emails it to the client automatically.

2.2 What problem it solves

Clients need to track content changes across websites over time without doing it manually. This system makes that fully automatic — no human needs to visit the site, read it, or decide what changed. The weekly email gives the client a clean digest of everything new that happened on their monitored websites that week.

2.3 Who uses it

Role	What they can do
Admin	Register clients and URLs, run the pipeline manually, view all reports and logs, stop monitoring, re-send failed emails
Viewer	Read-only access — view reports and monitoring logs, cannot change anything
Client (email recipient)	Receives the weekly email report automatically every Sunday. Does not log in.

3. Technology Stack

Every technology choice below is fixed for version 1.0. No substitutions without manager approval.

Layer	Technology	Why this choice
Frontend	React 18 + TypeScript + Tailwind CSS	Industry-standard SPA framework. TypeScript catches bugs at compile time.
Backend	ASP.NET Core Web API on .NET 10	Latest LTS release. C# is the team's required language.
ORM	Entity Framework Core 10 (Code-First)	Eliminates raw SQL for transactional data. Migrations keep schema in source control.
Primary DB	Microsoft SQL Server (via SSMS)	Transactional data: clients, URLs, logs, hashes.
Report / Analytics DB	DuckDB (embedded, no server)	Stores summaries and reports. Fast queries, zero infrastructure.
Browser automation	Microsoft Playwright for .NET	Handles static AND JavaScript-rendered sites. Officially maintained by Microsoft.
HTML parsing	AngleSharp (.NET)	Strips ads and noise from scraped HTML. Pure C#, no external process.
Summarization	Custom TF-IDF (pure C#, no API)	Fully offline. No API costs, no rate limits, no data leaving the machine.
Background jobs	Hangfire (SQL Server job store)	Persistent scheduled jobs that survive application restarts.
Authentication	ASP.NET Core Identity + JWT	Standard .NET auth pattern. Admin and Viewer roles.
PDF generation	QuestPDF (.NET)	Free for internal use. Reliable structured PDF output.
Word generation	Open XML SDK (.NET)	Microsoft's own library for .docx generation.
Email	MailKit (SMTP to Outlook)	Simple SMTP sender. No Azure dependency required for local deployment.
Logging	Serilog (file + console)	Structured logs with daily rolling files.
Deployment	Local machine — IIS or Kestrel self-hosted	v1.0 is local only. No cloud required.

4. The Daily Pipeline — How It Works

This is the most important section. Everything else in this document supports this pipeline. Read this first.

★ **Core design principle**
The content hash is the single source of truth for deciding whether a page has changed. The sitemap (when available) is used only to discover which URLs exist. It is NOT used to decide what changed. This applies equally to all websites, with or without a sitemap.

PHASE 1 — URL Discovery

The pipeline fires once per day for each active registered URL. The first job is to build a list of all pages that exist on that website today.

Step 1a — Check for a sitemap:

- Use Playwright to fetch the site's robots.txt and look for a Sitemap: directive.
- If not found, try these paths in order: /sitemap.xml, /sitemap_index.xml, /sitemap/sitemap.xml, /wp-sitemap.xml.
- If a sitemap index is found (a file containing links to child sitemaps), download all child sitemaps and merge their page URLs into one flat list.
- Record which track this URL uses (SITEMAP or HASH_BASED) in the ClientUrls table.

Step 1b — Track A: Sitemap found:

- Download the sitemap XML using Playwright's full browser rendering.
- Parse every <loc> element. This gives you the complete list of all pages on the site today.
- Save the XML file to disk for audit purposes: /storage/sitemaps/{clientId}/{urlId}/{YYYY-MM-DD}.xml

Step 1c — Track B: No sitemap found:

- Use Playwright to load the homepage.
- Extract all <a href='...'> internal links that belong to the same domain.
- Apply hard limits immediately: maximum 20 URLs, same domain only, filter to paths containing /blog, /article, /post, or /news (configurable).
- This is the URL list for today. Save discovered URLs to SQL Server.

End of Phase 1: Both tracks produce the same output — a flat list of page URLs to process today.

PHASE 2 — Per-URL Processing Loop

For every URL in the list produced by Phase 1, run these steps in order. Each URL is independent. A failure on one URL does not stop the others.

Step 2a — Scrape the page:

- Launch a Playwright headless Chromium browser and navigate to the URL.
- Wait for the page to fully load: networkidle state, maximum 30-second timeout.
- On failure: wait 2 seconds, retry. Wait 4 seconds, retry. Wait 8 seconds, retry.
- After 3 failures: write a WARNING to MonitoringLog, mark this URL as SCRAPE_FAILED in the log, and move to the next URL. The pipeline continues.

Step 2b — Clean the content (AngleSharp):

- Remove all <script>, <style>, <noscript>, <iframe>, <form>, <svg>, <canvas> elements.
- Remove structural chrome: <header>, <footer>, <nav>, <aside>.
- Remove elements whose class or id contains: ad, ads, advertisement, banner, promo, sidebar, cookie, popup, social, share, related, newsletter, subscribe.
- Extract the main body from the first matching selector: <article>, <main>, [role='main'], .post-content, .entry-content, .article-body, #content. Fallback: <body> after removals.
- Output: clean plain text. No HTML tags remain.

! **Why cleaning happens before hashing**
Raw HTML changes constantly — ad rotation, timestamps, cookie notices, visitor counters. If you hash raw HTML, the hash changes every day even when the actual article content is identical. By cleaning first and hashing the clean text, the hash only changes when real content changes.

Step 2c — Compute content hash:

- Compute SHA-256 of the clean text string.
- Look up the stored hash for this URL in the content_hashes SQL Server table.
- This applies to ALL URLs — whether discovered via sitemap (Track A) or homepage crawl (Track B). There is no exception.

Step 2d — Change decision:

- SAME hash as stored: no content change. Write a NO_CHANGE record to daily_summaries in DuckDB. Log INFO to MonitoringLog. Skip to next URL.
- DIFFERENT hash (or no stored hash = new URL): content has changed or page is new. Proceed to Step 2e.

Step 2e — Process changed content:

- Write the clean text to a Markdown (.md) file on disk. Store only the file PATH in DuckDB — never the file content itself.
- Run the TF-IDF summariser on the .md file (see Section 5 for algorithm detail).
- Store the summary text and .md file path in DuckDB page_summaries table.
- Update the stored hash in SQL Server content_hashes table with today's hash value and date.

PHASE 3 — Daily Summary Record

After all URLs in the list have been processed (or skipped), create one daily summary record in DuckDB:

- Date, client URL, total URLs discovered, pages changed and processed, pages with no change, pages that failed to scrape.
- Status: INITIAL_RUN (first ever run), CHANGED (at least one page changed), NO_CHANGE (nothing changed today), or PARTIAL (some pages failed).
- Compiled summary text: all individual page summaries for the day joined together in one readable block.

PHASE 4 — Weekly Report (Sundays only)

After Phase 3 completes, check whether today is Sunday. If not, the job ends and waits for the next daily run.

If today is Sunday:

1. Pull all daily_summaries records for the current weekly cycle from DuckDB.
2. Compile them into a structured Weekly Report document (see Section 7 for report format).
3. Generate a PDF version using QuestPDF.
4. Generate a Word (.docx) version using Open XML SDK.

5. Save both files to disk and store their paths in the weekly_reports DuckDB table.
6. Email the report to the client's registered email address via MailKit SMTP (Outlook).
7. On email failure: retry 3 times. After 3 failures, set email_status = FAILED in DuckDB. Surface a warning in the UI. Allow manual resend from the dashboard.
8. Mark the current weekly cycle as complete. The new cycle begins on Monday automatically.

> **Weekly cycle definition**
> Week 1 starts on the day the URL is first registered and ends the following Sunday. All subsequent weeks run Monday through Sunday. If a URL is registered on a Wednesday, Week 1 is a short partial week (Wednesday to Sunday). The weekly report for that short week is still generated and emailed on Sunday and is labelled 'Week 1 (Partial)'.

5. Local Summarisation — How It Works

The summariser is a pure C# class. It has no external dependencies, needs no internet connection, makes no API calls, and requires no NuGet packages beyond the standard .NET 10 runtime. It uses an algorithm called TF-IDF (Term Frequency — Inverse Document Frequency).

5.1 The algorithm, step by step

1. Read the .md file from disk.
2. Split the text into individual sentences (split on period-space, question mark-space, exclamation-space).
3. Tokenise each sentence into words. Remove stop words (a, an, the, is, are, was, were, it, in, on, at, to, of, and, or, but, etc.).
4. For each sentence, compute TF (Term Frequency) = count of each word divided by total words in that sentence.
5. Across all sentences, compute IDF (Inverse Document Frequency) = log(total sentences / sentences containing that word).
6. Score each sentence = sum of (TF × IDF) for every word in it.
7. Remove any sentence that contains advertisement keywords: click here, subscribe, buy now, limited offer, discount, promo, sign up.
8. Sort sentences by score, descending. Select the top N (default: 5, configurable).
9. Return the selected sentences in their original document order so the summary reads naturally.

5.2 Configuration

Setting	Key in appsettings.json	Default
Minimum sentences in summary	Summarizer:MinSentences	3
Maximum sentences in summary	Summarizer:MaxSentences	7
Stop words list (append to defaults)	Summarizer:AdditionalStopWords	(empty)

Quality expectation

> TF-IDF produces an extractive summary — it picks the most important existing sentences rather than rewriting them. Quality is good enough for a content monitoring digest. It will not produce publication-quality prose. This is the correct trade-off for a local-only, no-API system.

6. Database Design

6.1 SQL Server — Transactional Data

Managed by Entity Framework Core 10 Code-First migrations. All tables are created by running dotnet ef database update.

Clients

Column	Type	Notes
Id	int PK IDENTITY	Auto-incrementing primary key
FullName	nvarchar(200) NOT NULL	Client's full name
Email	nvarchar(256) NOT NULL UNIQUE	Delivery address for weekly reports
CompanyName	nvarchar(200) NULL	Optional
Phone	nvarchar(30) NULL	Optional
Status	nvarchar(20) NOT NULL	ACTIVE or INACTIVE (soft delete)
CreatedAt	datetime2 NOT NULL	Registration timestamp

ClientUrls

Column	Type	Notes
Id	int PK IDENTITY	
ClientId	int FK → Clients.Id	One client can have many URLs
Url	nvarchar(2048) NOT NULL	The registered URL
SitemapUrl	nvarchar(2048) NULL	Auto-detected on first run
TrackType	nvarchar(20) NOT NULL	SITEMAP or HASH_BASED
Status	nvarchar(20) NOT NULL	ACTIVE, PAUSED, STOPPED, SITEMAP_NOT_FOUND
RegisteredAt	datetime2 NOT NULL	
LastRunAt	datetime2 NULL	Updated after each pipeline run

Column	Type	Notes
CycleStartDate	date NULL	Start of current weekly cycle
UNIQUE CONSTRAINT	(ClientId, Url)	Prevents duplicate registrations

ContentHashes ← NEW in v3.0

★ **Why this table exists**

This is the core of the v3.0 architecture. Every scraped page has its clean-text SHA-256 hash stored here. The daily pipeline compares today's computed hash against the stored hash to decide whether to process or skip a page. This replaces XML diff as the change detection mechanism.

Column	Type	Notes
Id	int PK IDENTITY	
ClientUrlId	int FK → ClientUrls.Id	
PageUrl	nvarchar(2048) NOT NULL	The individual page URL
ContentHash	nvarchar(64) NOT NULL	SHA-256 hex string of clean text
LastHashDate	date NOT NULL	Date this hash was last computed
UNIQUE CONSTRAINT	(ClientUrlId, PageUrl)	One hash record per page

SitemapDownloads

Column	Type	Notes
Id	int PK IDENTITY	
ClientUrlId	int FK → ClientUrls.Id	
DownloadDate	date NOT NULL	
FilePath	nvarchar(500) NOT NULL	Local path to saved XML file
UrlCount	int	Number of <loc> entries found
Status	nvarchar(20)	SUCCESS or FAILED
CreatedAt	datetime2	

MonitoringLog

Column	Type	Notes
Id	int PK IDENTITY	
ClientUrlId	int FK NULL	NULL for system-level events
EventType	nvarchar(50)	SCHEDULER_RUN, SCRAPE, SUMMARIZE, REPORT_GEN, EMAIL_SEND, ERROR
Message	nvarchar(MAX)	
Level	nvarchar(20)	INFO, WARNING, ERROR
Timestamp	datetime2 NOT NULL	

AspNetUsers / Identity tables

Standard ASP.NET Core Identity tables generated automatically by EF Core Identity migrations. Provides user accounts, password hashing, and role management. No custom columns added in v1.0.

6.2 DuckDB — Report Analytics

Setup
DuckDB runs embedded inside the .NET process. Install via NuGet: DuckDB.NET.Data. The database file lives at {StorageBasePath}/analytics/blogmonitor.duckdb. No server installation needed.

page_summaries

Column	Type	Notes
id	BIGINT PK	
client_url_id	INTEGER NOT NULL	
page_url	VARCHAR NOT NULL	
page_title	VARCHAR	From <title> or first <h1>
summary_date	DATE NOT NULL	
week_number	INTEGER	
md_file_path	VARCHAR	Path to .md file on disk. NOT the file content.
summary_text	VARCHAR	The 3–7 sentence TF-IDF summary
word_count	INTEGER	Word count of clean extracted text

Column	Type	Notes
summary_status	VARCHAR	SUCCESS or SUMMARY_FAILED
created_at	TIMESTAMP	

daily_summaries

Column	Type	Notes
id	BIGINT PK	
client_url_id	INTEGER NOT NULL	
summary_date	DATE NOT NULL	
day_in_cycle	INTEGER	1 = registration day, increments daily
status	VARCHAR	INITIAL_RUN, CHANGED, NO_CHANGE, PARTIAL
pages_changed	INTEGER	URLs where hash differed
pages_no_change	INTEGER	URLs where hash matched
pages_failed	INTEGER	URLs that could not be scraped
compiled_summary	VARCHAR	All page summaries joined for this day
created_at	TIMESTAMP	

weekly_reports

Column	Type	Notes
id	BIGINT PK	
client_url_id	INTEGER NOT NULL	
client_id	INTEGER NOT NULL	
week_start_date	DATE	
week_end_date	DATE	Always a Sunday
week_number	INTEGER	
cycle_label	VARCHAR	e.g. 'Week 1 (Partial)', 'Week 2'
report_json	VARCHAR	Full report as structured JSON
md_file_path	VARCHAR	Path to Markdown version
pdf_file_path	VARCHAR	Path to PDF version

Column	Type	Notes
docx_file_path	VARCHAR	Path to Word version
total_days	INTEGER	
days_changed	INTEGER	
days_no_change	INTEGER	
total_pages_scraped	INTEGER	
email_status	VARCHAR	PENDING, SENT, FAILED
email_sent_at	TIMESTAMP NULL	
created_at	TIMESTAMP	

7. Complete Feature List

7.1 Client Management

- Create, view, edit, and soft-delete client records (Admin only).
- Each client has: full name, email (used for weekly report delivery), company name, phone, status.
- Soft delete sets status to INACTIVE. No data is permanently deleted.
- A client may own zero or many monitored URLs.

7.2 URL Registration & Management

- Admin selects a client and registers one or more URLs for that client.
- System validates the URL format and checks reachability before saving.
- Duplicate URLs per client are blocked by a database constraint.
- Admin can stop monitoring a URL at any time via a 'Stop Fetching Data' button with confirmation dialog.
- Stopping a URL removes its Hangfire job, sets status to STOPPED, and generates a final partial weekly report if a cycle is in progress.
- Admin can manually trigger a pipeline run per URL for testing ('Run Now' button).

7.3 Authentication

- Users log in with email and password. Passwords hashed with PBKDF2 via ASP.NET Core Identity.
- On login: API issues a JWT access token (60-minute expiry) and sets a refresh token in an HttpOnly cookie (7-day expiry).
- Access token stored in browser memory only — never in localStorage.
- Two roles: Admin (full access) and Viewer (read-only).
- All API endpoints except /api/auth/login and /api/auth/register require a valid JWT.

7.4 Automated Daily Pipeline

- Hangfire runs the pipeline daily at 02:00 AM (configurable cron expression).
- Each active URL has its own independent Hangfire recurring job.

- Pipeline runs Phase 1 through Phase 4 as described in Section 4.
- All execution events are written to MonitoringLog in SQL Server.

7.5 Weekly Report

- Generated every Sunday at the end of the pipeline run.
- Contains: report header, executive summary statistics, day-by-day breakdown with summaries, appendix of all URLs processed.
- Saved in three formats: Markdown (.md), PDF, Word (.docx).
- Emailed to the client's registered email via MailKit SMTP to Outlook.
- Viewable in-browser in the Report Viewer page of the React UI.

7.6 Report Download

- From the Report Viewer page, Admin and Viewer can download any weekly report.
- Two download buttons per report: Download as PDF and Download as Word.
- The backend streams the pre-generated file from disk with the correct Content-Type header.

7.7 UI Pages

Page	Access	Purpose
Login / Register	/login, /register	Public. JWT stored in memory on login.
Dashboard	Admin + Viewer	Summary cards per client: active URLs, last run status, latest report link.
URL Manager	Admin + Viewer	Table of all registered URLs with status, last run, week day number, actions.
Report Viewer	Admin + Viewer	Browse weekly reports by client and URL. View in-browser, download PDF or Word, email status badge.
Monitoring Log	Admin only	Paginated log table filtered by URL and date. Colour-coded by severity.
Client Manager	Admin only	CRUD for client records.

8. API Endpoints

8.1 Authentication

Method	Endpoint	Auth	What it does
POST	/api/auth/register	Public	Create account. Body: { email, password, role }
POST	/api/auth/login	Public	Login. Returns JWT. Sets refresh cookie.
POST	/api/auth/refresh	Cookie only	Get new access token using refresh cookie.
POST	/api/auth/logout	JWT	Revoke refresh token. Clear cookie.

8.2 Clients

Method	Endpoint	Auth	What it does
GET	/api/clients	Admin	List all clients (paginated).
POST	/api/clients	Admin	Create client. Body: { fullName, email, companyName?, phone? }
GET	/api/clients/{id}	Admin	Get one client.
PUT	/api/clients/{id}	Admin	Update client details.
DELETE	/api/clients/{id}	Admin	Soft-delete (sets status=INACTIVE).

8.3 URLs

Method	Endpoint	Auth	What it does
GET	/api/urls	Admin+Viewer	List all registered URLs with status.
POST	/api/urls	Admin	Register URL. Body: { clientId, url }
GET	/api/urls/{id}	Admin+Viewer	Get URL details.
PUT	/api/urls/{id}/stop	Admin	Stop monitoring. Removes Hangfire job.
POST	/api/urls/{id}/run	Admin	Manually trigger pipeline now.

8.4 Reports

Method	Endpoint	Auth	What it does
GET	/api/reports/weekly	Admin+Viewer	Query by clientUrlId. List weekly reports.

Method	Endpoint	Auth	What it does
GET	/api/reports/weekly/{id}	Admin+Viewer	Get full report JSON.
GET	/api/reports/weekly/{id}/download	Admin+Viewer	?format=pdf or docx. Streams file.
POST	/api/reports/weekly/{id}/resend	Admin	Re-trigger email for a FAILED report.
GET	/api/reports/daily	Admin+Viewer	?clientUrlId, ?date. Get daily summary.

8.5 Logs

Method	Endpoint	Auth	What it does
GET	/api/logs	Admin	?clientUrlId, ?level, ?from, ?to, ?page, ?pageSize. Paginated logs.

9. File Storage Conventions

All paths are under a configurable base directory set in appsettings.json as Storage:BasePath. Default: C:\BlogMonitorStorage\ on Windows.

File type	Path pattern	Example
Sitemap XML	{base}/sitemaps/{clientId}/{urlId}/{YYYY-MM-DD}.xml	/sitemaps/1/3/2025-06-22.xml
Page Markdown	{base}/markdown/{clientId}/{urlId}/{YYYY-MM-DD}/{slug}.md	/markdown/1/3/2025-06-22/intro-to-react.md
Weekly report MD	{base}/reports/{clientId}/{urlId}/{YYYY}-W{WW}.md	/reports/1/3/2025-W26.md
Weekly report PDF	{base}/reports/{clientId}/{urlId}/{YYYY}-W{WW}.pdf	/reports/1/3/2025-W26.pdf
Weekly report DOCX	{base}/reports/{clientId}/{urlId}/{YYYY}-W{WW}.docx	/reports/1/3/2025-W26.docx
DuckDB database	{base}/analytics/blogmonitor.duckdb	Single file, no server
Serilog logs	{base}/logs/app-{YYYY-MM-DD}.log	Daily rolling log files

Slug generation: take the URL path segment, lowercase it, replace non-alphanumeric characters with hyphens, trim leading/trailing hyphens, truncate to 100 characters.

All files are retained indefinitely. No automatic deletion. No storage quota in v1.0.

10. Edge Cases & How They Are Handled

Scenario	What the system does
First ever run for a URL (Day 1 / INITIAL_RUN)	No stored hashes exist yet. Scrape every URL discovered. Compute and store a hash for each. Mark daily summary as INITIAL_RUN. This becomes the baseline for all future comparisons.
Page content unchanged (hash matches)	Log NO_CHANGE for this URL. Skip scraping and summarisation. The day still appears in the weekly report with its NO_CHANGE status — the report is always complete.
Page content changed (hash differs)	Process this URL fully: clean → .md → TF-IDF → store summary → update hash.
New page appears (no stored hash)	Treat as changed. Process fully. Store hash.
Page disappears from sitemap	Log that the URL was removed. No scraping needed. Hash record remains but is not updated.
Sitemap not found	Flag ClientUrl.Status = SITEMAP_NOT_FOUND. Switch to Track B (homepage crawl). Log WARNING.
Page scrape times out after 3 retries	Log WARNING. Skip this URL. Continue with the next. Pipeline never stops for one bad page.
Extracted content too short (< 50 words)	Skip summarisation. Set summary_status = SUMMARY_FAILED. Log WARNING. Store hash anyway.
Sunday: no changes all week	Weekly report still generated. Each day shows NO_CHANGE status. Statistics reflect zero pages scraped. Report is emailed normally.
URL stopped mid-week	Generate a partial weekly report for the incomplete cycle. Label it 'Week N (Partial — Stopped)'. Email it. Remove Hangfire job.
Email send fails after 3 retries	Set email_status = FAILED in DuckDB. Show warning badge in UI. Admin can click 'Resend' manually.
Duplicate URL registration	UNIQUE constraint (ClientId, Url) in SQL Server rejects it. API returns 409 Conflict with a clear error message.
Track B: homepage has 500+ links	Hard cap of 20 URLs applied immediately. Only same-domain, blog-path links are included. Configurable in appsettings.json.
Sitemap index file (contains child sitemaps)	System recursively downloads and parses all child sitemaps. All <loc> entries merged into one deduplicated list.

11. Non-Functional Requirements

ID	Category	Requirement	Priority
NFR-01	Performance	All API responses return within 2 seconds under single-user local load.	High
NFR-02	Performance	Background pipeline runs fully async. Never blocks the main API thread.	High
NFR-03	Security	All secrets (JWT key, SMTP credentials, DB connection string) stored in .NET User Secrets (dev) or environment variables (production). Never committed to source control.	High
NFR-04	Security	Passwords hashed with PBKDF2 via ASP.NET Core Identity. No plaintext storage.	High
NFR-05	Security	JWT access token: 60-minute expiry. Refresh token: 7-day expiry in HttpOnly cookie.	High
NFR-06	Reliability	Scrape failures retry up to 3 times with exponential backoff (2s, 4s, 8s) before logging and skipping.	High
NFR-07	Reliability	Email failures caught and logged. UI surfaces the failure. Manual resend available.	High
NFR-08	Maintainability	All services implement C# interfaces (ISitemapDownloader, IContentScraper, ITfidfSummariser, IEmailSender, IReportGenerator) for dependency injection and unit testability.	Medium
NFR-09	Maintainability	All configurable values (cron, paths, delays, sentence counts, crawl limits) live in appsettings.json. No hard-coded values.	Medium
NFR-10	Observability	Serilog structured logging. Daily rolling log files. Log levels: DEBUG, INFO, WARNING, ERROR.	Medium
NFR-11	Observability	Swagger UI available at /swagger in Development environment only.	Low
NFR-12	Usability	React UI responsive on 1280px+ desktop and 768px+ tablet.	Low
NFR-13	Scalability	System supports at least 10 concurrently monitored URLs without pipeline conflicts. DuckDB writes serialised via a semaphore lock.	Medium

12. What Changed from v2.0 to v3.0

This section exists specifically for the Engineering Manager's review. If you approved v2.0, this table shows exactly what changed and why.

Area	v2.0 approach	v3.0 change	Reason
Change detection	XML diff used to detect changed pages	Hash-based. XML diff now used only for URL discovery.	XML diff tells you which pages exist, not whether their content changed. A blog can update an article without changing its sitemap entry. Hash solves this.
Hash scope	Hash applied only in Track B (no-sitemap sites)	Hash applied to ALL URLs in both tracks equally.	Inconsistent change detection. Track A sites could miss content updates.
Hash input	Hash computed on raw scraped HTML	Hash computed on clean text (after AngleSharp processing).	Raw HTML changes daily due to ads, timestamps, cookie banners. Hashing raw HTML produces false positives.
Track B crawl limit	No limit specified	Hard cap: max 20 URLs, same domain, blog-path filter.	Without a limit, Track B could collect thousands of links from a large homepage.
New SQL table	No content hash storage table	ContentHashes table added to SQL Server.	The hash comparison needs persistent storage. This table is the core of v3.0.
.md storage in DuckDB	File content stored as TEXT column	File PATH stored, content stays on disk.	Storing large text blobs in DuckDB inflates the database and duplicates data already on disk.

13. Configuration Reference (appsettings.json)

All configurable values and their keys. Sensitive values (marked *) must be stored in .NET User Secrets, not in appsettings.json.

Key	Default	Description
ConnectionStrings:SqlServer *	(no default)	SQL Server connection string
ConnectionStrings:DuckDb	Storage:BasePath + /analytics/blogmonitor.duckdb	DuckDB file path
Jwt:SecretKey *	(no default)	JWT signing key — minimum 32 characters
Jwt:AccessTokenExpiryMinutes	60	Access token lifetime
Jwt:RefreshTokenExpiryDays	7	Refresh token lifetime
Email:SmtpHost	smtp-mail.outlook.com	Outlook SMTP server
Email:SmtpPort	587	STARTTLS port
Email:SenderEmail *	(no default)	Outlook sender address
Email:SenderPassword *	(no default)	Outlook app password
Storage:BasePath	C:\BlogMonitorStorage	Root directory for all files
Scheduler:DailyCron	0 2 * * *	2:00 AM daily
Scraper:PageTimeoutSeconds	30	Per-page Playwright timeout
Scraper:DelayBetweenPagesSeconds	2	Polite delay between page loads
Scraper:MaxRetries	3	Retry attempts before skipping a page
Scraper:TrackBMaxUrls	20	Maximum URLs collected in Track B
Scraper:TrackBPathFilter	/blog,/article,/post,/news	Comma-separated path prefixes for Track B
Summarizer:MinSentences	3	Minimum sentences in a summary
Summarizer:MaxSentences	7	Maximum sentences in a summary
Summarizer:MinContentWords	50	Skip summarisation if clean text is shorter than this

14. Glossary

Term	Plain English definition
Sitemap XML	A file most websites publish at /sitemap.xml listing every page URL on the site. The system downloads this daily to know which pages exist.
Track A	The pipeline path for websites that have a sitemap. URLs are discovered from the sitemap XML.
Track B	The pipeline path for websites without a sitemap. URLs are discovered by crawling internal links on the homepage.
SHA-256 hash	A fingerprint of a piece of text. If the text changes even slightly, the fingerprint changes completely. Used here to detect whether a page's content has changed since yesterday.
Clean text	The article body text after AngleSharp has removed all HTML tags, ads, navigation, scripts, and other noise. This is what gets hashed and summarised.
TF-IDF	Term Frequency — Inverse Document Frequency. A mathematical scoring method that gives higher scores to sentences containing words that are rare across the document but frequent in that specific sentence — i.e., the most informative sentences.
Extractive summary	A summary built by selecting the highest-scoring existing sentences from the original text. Contrasted with abstractive summarisation (rewriting in new words), which would require an AI language model.
DuckDB	An embedded database that runs inside the .NET application with no separate server. Used to store and query daily and weekly summaries.
Hangfire	A .NET library for running background jobs on a schedule. Each registered URL gets its own recurring Hangfire job that fires the daily pipeline.
Weekly cycle	Week 1 starts on the URL's registration day and ends the following Sunday. All subsequent weeks run Monday through Sunday.
INITIAL_RUN	The status of the first daily summary for a URL. On this run, all discovered pages are scraped and hashed with no comparison — they become the baseline.
JWT	JSON Web Token. A signed, time-limited token issued on login that the browser sends with every API request to prove the user is authenticated.
EF Core Code-First	An approach where you define your database tables as C# classes, and Entity Framework Core generates and runs the SQL migration scripts automatically.

15. Feasibility Assessment

VERDICT: FEASIBLE — RECOMMENDED TO PROCEED

The v3.0 architecture addresses all loopholes identified in peer review. The stack is mature and proven. The project is well-scoped for a local deployment by an intern developer.

15.1 Risk register

Risk	Likelihood	Impact	Mitigation
Target site blocks Playwright (Cloudflare, bot detection)	Medium	Medium	Add realistic User-Agent headers and configurable delays. Accept that ~5–10% of heavily protected sites may not scrape. Log and skip — do not crash.
Weekly cycle date arithmetic bugs (partial weeks, year boundaries)	Medium	High	Write unit tests for WeeklyCycleService before building the scheduler. Test: registration on Wednesday, registration on Sunday, registration on Monday, registration on Dec 30.
Playwright Chromium binaries unavailable on restricted networks	Low	Medium	Run 'playwright install chromium' during setup. Document the command in the README. Pre-download if needed and set PLAYWRIGHT_BROWSERS_PATH.
DuckDB concurrent write conflicts (multiple Hangfire jobs)	Low	Medium	Use a single DuckDB connection wrapped in a SemaphoreSlim(1) lock in the service layer. Serialize all writes.
Outlook SMTP authentication failure	Medium	Low	Use an Outlook app password if 2FA is enabled on the sender account. Document setup steps clearly.
Track B crawl returns irrelevant pages	Low	Low	Configure Scraper:TrackBPathFilter to include only blog/article paths for the specific site. This is per-site configuration.

15.2 Recommended build order

Build in this order to validate the riskiest components first:

1. Set up SQL Server, run EF Core migrations, verify all tables are created correctly.
2. Install Playwright. Write a standalone test that scrapes a real blog URL and prints the clean text to console. Verify it works before writing any application code.
3. Build and unit-test the TF-IDF summariser class in complete isolation with real blog post text.
4. Build and unit-test WeeklyCycleService with the date arithmetic test cases listed above.

5. Implement ASP.NET Core Identity + JWT auth. Test login/refresh/logout with Postman.
6. Wire the pipeline together (Phase 1 → 2 → 3) without scheduling. Trigger it manually via a test endpoint.
7. Add Hangfire scheduling. Verify the job persists across application restarts.
8. Build the React UI.
9. Add email delivery last — test with a real Outlook account.

— End of Software Requirements Specification v3.0 —

FINAL | Version 3.0 | Internal Use Only | Ready for Development